Password-based authentication is used widely by operating systems as well as databases. However, the use of passwords has some drawbacks, especially over a network. If an eavesdropper is able to "sniff" the data being sent over the network, she may be able to find the password as it is being sent across the network. Once the eavesdropper has a user name and password, she can connect to the database, pretending to be the legitimate user.

A more secure scheme involves a **challenge-response** system. The database system sends a challenge string to the user. The user encrypts the challenge string using a secret password as encryption key, and then returns the result. The database system can verify the authenticity of the user by decrypting the string with the same secret password, and checking the result with the original challenge string. This scheme ensures that no passwords travel across the network.

Public-key systems can be used for encryption in challenge–response systems. The database system encrypts a challenge string using the user's public key and sends it to the user. The user decrypts the string using her private key, and returns the result to the database system. The database system then checks the response. This scheme has the added benefit of not storing the secret password in the database, where it could potentially be seen by system administrators.

Another interesting application of public-key encryption is in **digital signatures** to verify authenticity of data; digital signatures play the electronic role of physical signatures on documents. The private key is used to sign data, and the signed data can be made public. Anyone can verify them by the public key, but no one could have generated the signed data without having the private key. Thus, we can **authenticate** the data; that is, we can verify that the data were indeed created by the person who claims to have created them.

Furthermore, digital signatures also serve to ensure **nonrepudiation**. That is, in case the person who created the data later claims she did not create it (the electronic equivalent of claiming not to have signed the check), we can prove that that person must have created the data (unless her private key was leaked to others).

6.8 Summary

- Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency.
- In earlier chapters, we considered several forms of constraints, including key declarations and the declaration of the form of a relationship (many to many, many to one, one to one). In this chapter, we considered several additional forms of constraints, and discussed mechanisms for ensuring the maintenance of these constraints.
- Domain constraints specify the set of possible values that may be associated with an attribute. Such constraints may also prohibit the use of null values for particular attributes.

- Referential-integrity constraints ensure that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation.
- Domain constraints, and referential-integrity constraints are relatively easy to test. Use of more complex constraints may lead to substantial overhead. We saw two ways to express more general constraints. Assertions are declarative expressions that state predicates that we require always to be true.
- Triggers define actions to be executed automatically when certain events occur and corresponding conditions are satisfied. Triggers have many uses, such as implementing business rules, audit logging, and even carrying out actions outside the database system. Although triggers were added only lately to the SQL standard as part of SQL:1999, most database systems have long implemented triggers.
- The data stored in the database need to be protected from unauthorized access, malicious destruction or alteration, and accidental introduction of inconsistency.
- It is easier to protect against accidental loss of data consistency than to protect against malicious access to the database. Absolute protection of the database from malicious abuse is not possible, but the cost to the perpetrator can be made sufficiently high to deter most, if not all, attempts to access the database without proper authority.
- A user may have several forms of authorization on parts of the database. Authorization is a means by which the database system can be protected against malicious or unauthorized access.
- A user who has been granted some form of authority may be allowed to pass on this authority to other users. However, we must be careful about how authorization can be passed among users if we are to ensure that such authorization can be revoked at some future time.
- Roles help to assign a set of privileges to a user according to on the role that the user plays in the organization.
- The various authorization provisions in a database system may not provide sufficient protection for highly sensitive data. In such cases, data can be *encrypted*. Only a user who knows how to decipher (*decrypt*) the encrypted data can read them. Encryption also forms the basis for secure authentication of users.

Review Terms

- Domain constraints
- Check clause
- Referential integrity
- Primary key constraint
- Unique constraint
- Foreign key constraint

- Cascade
- Assertion
- Trigger
- Event-condition-action model
- Before and after triggers
- Transition variables and tables
- Database security
- Levels of security
- Authorization
- Privileges
  - ☐ Read
  - ☐ Insert
  - ☐ Update
  - ☐ Delete
  - ☐ Index
  - ☐ Resource
  - ☐ Alteration
  - ☐ Drop
  - ☐ Grant
  - ☐ All privileges
- Authorization graph
- Granting of privileges
- Roles
- Encryption
- Secret-key encryption
- Public-key encryption
- Authentication
- Challenge–response system
- Digital signature
- Nonrepudiation

Exercises

6.1 Complete the SQL DDL definition of the bank database of Figure 6.2 to include the relations *loan* and *borrower*.

6.2 Consider the following relational database:

employee (*employee-name*, *street*, *city*)
works (*employee-name*, *company-name*, *salary*)
company (*company-name*, *city*)
manages (*employee-name*, *manager-name*)

Give an SQL DDL definition of this database. Identify referential-integrity constraints that should hold, and include them in the DDL definition.

6.3 Referential-integrity constraints as defined in this chapter involve exactly two relations. Consider a database that includes the following relations:

salaried-worker (*name, office, phone, salary*)
hourly-worker (*name, hourly-wage*)
address (*name, street, city*)

Suppose that we wish to require that every name that appears in *address* appear in either *salaried-worker* or *hourly-worker*, but not necessarily in both.

a. Propose a syntax for expressing such constraints.
b. Discuss the actions that the system must take to enforce a constraint of this form.

6.4 SQL allows a foreign-key dependency to refer to the same relation, as in the following example:

```
create table manager
    (employee-name    char(20)  not null
     manager-name     char(20)  not null,
     primary key employee-name,
     foreign key (manager-name) references manager
                                    on delete cascade )
```

Here, *employee-name* is a key to the table *manager*, meaning that each employee has at most one manager. The foreign-key clause requires that every manager also be an employee. Explain exactly what happens when a tuple in the relation *manager* is deleted.

6.5 Suppose there are two relations r and s, such that the foreign key B of r references the primary key A of s. Describe how the trigger mechanism can be used to implement the **on delete cascade** option, when a tuple is deleted from s.

6.6 Write an assertion for the bank database to ensure that the assets value for the Perryridge branch is equal to the sum of all the amounts lent by the Perryridge branch.

6.7 Write an SQL trigger to carry out the following action: On **delete** of an account, for each owner of the account, check if the owner has any remaining accounts, and if she does not, delete her from the *depositor* relation.

6.8 Consider a view *branch-cust* defined as follows:

```
create view branch-cust as
    select branch-name, customer-name
    from depositor, account
    where depositor.account-number = account.account-number
```

Suppose that the view is *materialized*, that is, the view is computed and stored. Write active rules to *maintain* the view, that is, to keep it up to date on insertions to and deletions from *depositor* or *account*. Do not bother about updates.

6.9 Make a list of security concerns for a bank. For each item on your list, state whether this concern relates to physical security, human security, operating-system security, or database security.

6.10 Using the relations of our sample bank database, write an SQL expression to define the following views:

a. A view containing the account numbers and customer names (but not the balances) for all accounts at the Deer Park branch.

b. A view containing the names and addresses of all customers who have an account with the bank, but do not have a loan.

c. A view containing the name and average account balance of every customer of the Rock Ridge branch.

6.11 For each of the views that you defined in Exercise 6.10, explain how updates would be performed (if they should be allowed at all). *Hint*: See the discussion of views in Chapter 3.

6.12 In Chapter 3, we described the use of views to simplify access to the database by users who need to see only part of the database. In this chapter, we described the use of views as a security mechanism. Do these two purposes for views ever conflict? Explain your answer.

6.13 What is the purpose of having separate categories for index authorization and resource authorization?

6.14 Database systems that store each relation in a separate operating-system file may use the operating system's security and authorization scheme, instead of defining a special scheme themselves. Discuss an advantage and a disadvantage of such an approach.

6.15 What are two advantages of encrypting data stored in the database?

6.16 Perhaps the most important data items in any database system are the passwords that control access to the database. Suggest a scheme for the secure storage of passwords. Be sure that your scheme allows the system to test passwords supplied by users who are attempting to log into the system.

Bibliographical Notes

Discussions of integrity constraints in the relational model are offered by Hammer and McLeod [1975], Stonebraker [1975], Eswaran and Chamberlin [1975], Schmid and Swenson [1975] and Codd [1979]. The original SQL proposals for assertions and triggers are discussed in Astrahan et al. [1976], Chamberlin et al. [1976], and Chamberlin et al. [1981]. See the bibliographic notes of Chapter 4 for references to SQL standards and books on SQL.

Discussions of efficient maintenance and checking of semantic-integrity assertions are offered by Hammer and Sarin [1978], Badal and Popek [1979], Bernstein et al. [1980a], Hsu and Imielinski [1985], McCune and Henschen [1989], and Chomicki [1992]. An alternative to using run-time integrity checking is certifying the correctness of programs that access the database. Sheard and Stemple [1989] discusses this approach.

Active databases are databases that support triggers and other mechanisms that permit the database to take actions on occurrence of events. McCarthy and Dayal [1989] discuss the architecture of an active database system based on the event–condition–action formalism. Widom and Finkelstein [1990] describe the architecture of a rule system based on set-oriented rules; the implementation of the rule system

on the Starburst extensible database system is presented in Widom et al. [1991]. Consider an execution mechanism that allows a nondeterministic choice of which rule to execute next. A rule system is said to be **confluent** if, regardless of the rule chosen, the final state is the same. Issues of termination, nondeterminism, and confluence of rule systems are discussed in Aiken et al. [1995].

Security aspects of computer systems in general are discussed in Bell and LaPadula [1976] and by US Dept. of Defense [1985]. Security aspects of SQL can be found in the SQL standards and textbooks on SQL referenced in the bibliographic notes of Chapter 4.

Stonebraker and Wong [1974] discusses the Ingres approach to security, which involves modification of users' queries to ensure that users do not access data for which authorization has not been granted. Denning and Denning [1979] survey database security.

Database systems that can produce incorrect answers when necessary for security maintenance are discussed in Winslett et al. [1994] and Tendick and Matloff [1994]. Work on security in relational databases includes that of Stachour and Thuraisingham [1990], Jajodia and Sandhu [1990], and Qian and Lunt [1996]. Operating-system security issues are discussed in most operating-system texts, including Silberschatz and Galvin [1998].

Stallings [1998] provides a textbook description of cryptography. Daemen and Rijmen [2000] present the Rijndael algorithm. The Data Encryption Standard is presented by US Dept. of Commerce [1977]. Public-key encryption is discussed by Rivest et al. [1978]. Other discussions on cryptography include Diffie and Hellman [1979], Simmons [1979], Fernandez et al. [1981], and Akl [1983].